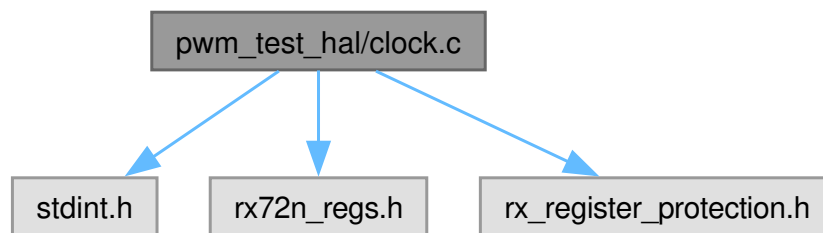

>>

>>

>>

```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
```



*

>>

>>>

enum [clock_byte_constants_t](#) : uint8_t

	>

```
00075         : uint8_t {
00076   k_hococr_running  = 0x00U, /**< HOCOCR: 0 = HOCO running */
00077   k_pllcr2_enable   = 0x00U, /**< PLLCR2: 0 = PLL running (inverted logic) */
00078   k_memwait_one     = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00079   k_oscovfsr_hcovf  = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00080   k_oscovfsr_plovf  = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00081 } clock_byte_constants_t;
```

```
enum clock_constants_word_t : uint16_t
```


```
00043         : uint16_t {
00044     /*
00045     * PLLCR layout (16 bits):
00046     * bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00047     * bit [4] PLLSRCSEL: 0=MOSC, 1=HOCO
00048     * bits [13:8] STC : multiply = (STC + 1) / 2
00049     *
00050     * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00051     * STC = (12 * 2) - 1 = 23 = 0x17.
00052     * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00053     */
00054     k_pllcr_hoco_x12 = 0x1710U,
00055
00056     /*
00057     * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00058     * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00059     */
00060     k_sckcr2_uck_div4 = 0x0031U,
00061
00062     /* PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00063     k_packcr_pll_source = 0x0000U,
00064 } clock_constants_word_t;
```

```
enum clock_extra_addrs_t : uintptr_t
```

--	--

```
00036         : uintptr_t {
00037     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00038 } clock_extra_addrs_t;
```

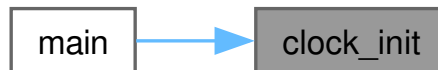
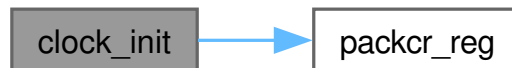
```
enum clock_sckcr_t : uint32_t
```

--	--

```
00066         : uint32_t {
00067     /*
00068     * SCKCR dividers:
00069     * FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00070     * PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00071     */
00072     k_sckcr_value = 0x21C21211U,
00073 } clock_sckcr_t;
```

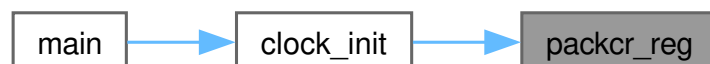
```
void clock_init (
    void )
```

```
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscofsr & k_oscofsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscofsr & k_oscofsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->sckcr = k_sckcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }
```



```
volatile uint16_t * packcr_reg (
    void ) [inline], [static]
```

```
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
```



```

00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.
00004  *
00005  * @details
00006  * This test sources the PLL from the internal HOCO (16 MHz) instead of
00007  * the STAR PCB's 24 MHz external crystal on MOSC/EXTAL. The choice is
00008  * deliberate (isolate the PLL bring-up from crystal startup), not because
00009  * the board lacks a crystal -- it has one.
00010  *
00011  * Target clocks:
00012  * - PLL = HOCO 16 MHz x 12 = 192 MHz
00013  * - ICLK = 192 / 2 = 96 MHz (CPU)
00014  * - PCKA = 192 / 2 = 96 MHz
00015  * - PCKB = 192 / 4 = 48 MHz (CMT0 ThreadX tick)
00016  * - PCKC = 192 / 2 = 96 MHz
00017  * - PCKD = 192 / 2 = 96 MHz
00018  * - FCLK = 192 / 4 = 48 MHz (flash)
00019  * - BCLK = 192 / 4 = 48 MHz
00020  * - UCLK = 192 / 4 = 48 MHz (not used, but set for completeness)
00021  *
00022  * Path: HOCO 16 MHz -> PLIDIV /1 -> STC x12 -> 192 MHz PLL output.
00023  *
00024  * SPDX-License-Identifier: MIT
00025  * @copyright Copyright (c) 2026 Locked Inc.
00026  */
00027
00028 #include <stdint.h>
00029
00030 #include "rx72n_regs.h"
00031 #include "rx_register_protection.h"
00032
00033 /**
=====
00034  * Register addresses not exposed by the rx_system_regs_t struct
00035  *
=====
00036  */
00037 typedef enum : uintptr_t {
00038     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00039     clock_extra_addrs_t;
00040 }
00041
00042 /**
=====
00043  * Clock configuration constants
00044  *
=====
00045  */
00046 typedef enum : uint16_t {
00047     /**
00048      * PLLCR layout (16 bits):
00049      * bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00050      * bit [4] PLLSRCSEL: 0=MOSC, 1=HOCO
00051      * bits [13:8] STC : multiply = (STC + 1) / 2
00052      *
00053      * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00054      * STC = (12 * 2) - 1 = 23 = 0x17.
00055      * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00056      */
00057     k_pllcr_hoco_x12 = 0x1710U,
00058
00059     /**
00060      * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00061      * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00062      */
00063     k_sckcr2_uck_div4 = 0x0031U,
00064
00065     /** PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00066     k_packcr_pll_source = 0x0000U,
00067 } clock_constants_word_t;
00068
00069 typedef enum : uint32_t {
00070     /**
00071      * SCKCR dividers:
00072      * FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00073      * PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00074      */
00075     k_sckcr_value = 0x21C21211U,
00076 } clock_sckcr_t;
00077
00078 typedef enum : uint8_t {
00079     k_hococr_running = 0x00U, /**< HOCOCR: 0 = HOCO running */
00080     k_pllcr2_enable = 0x00U, /**< PLLCR2: 0 = PLL running (inverted logic) */
00081     k_memwait_one = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00082     k_oscovfsr_hcovf = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00083     k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */

```

```

00081 } clock_byte_constants_t;
00082
00083 /*
=====
00084 * Inline accessors
00085 *
=====
*/
00086 static inline volatile uint16_t *packcr_reg(void)
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
00090
00091 /*
=====
00092 * clock_init -- bring the chip from LOCO 240 kHz to PLL 192/96/48 MHz.
00093 *
00094 * Uses HOCO (16 MHz internal oscillator) as the PLL source.
00095 * No external crystal required.
00096 *
00097 * Must run before any peripheral that depends on PCKB is touched.
00098 * Called from main() before cmt0_init() and before tx_kernel_enter().
00099 *
=====
*/
00100 void clock_init(void)
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->sckcr = k_sckcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }

00001 /*
00002 * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003 *
00004 * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005 * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006 */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00012 }
00013
00014 SECTIONS

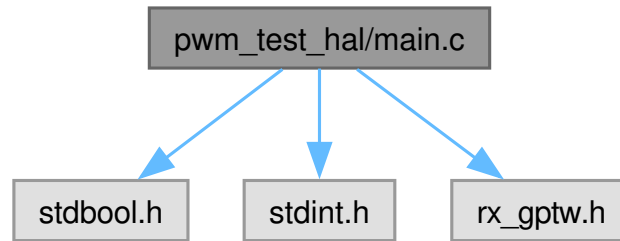
```

```

00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P)
00022         *(.rodata*)
00023         *(C)
00024         *(C)
00025         *(C)
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030     * Relocatable vector table -- startup.S loads INTB with this address.
00031     * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032     * used, but allocate 256 entries (1024 bytes) for safety.
00033     */
00034     .rvecvectors ALIGN(4) :
00035     {
00036         _rvecvectors_start = .;
00037         KEEP(*(rvecvectors))
00038         _rvecvectors_end = .;
00039     } > ROM
00040
00041     /*
00042     * Initialised data -- LMA stays in ROM, VMA in RAM.
00043     * startup.S copies _data..edata from _mdata on boot.
00044     */
00045     _mdata = .;
00046     .data : AT(_mdata)
00047     {
00048         data = .;
00049         *(.data*)
00050         *(D)
00051         *(D)
00052         *(D)
00053         _edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         _bss = .;
00060         *(.bss*)
00061         *(COMMON)
00062         *(B)
00063         *(B)
00064         *(B)
00065         _ebss = .;
00066         . = ALIGN(4);
00067         _end = .;
00068     } > RAM AT>RAM
00069
00070     /*
00071     * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072     */
00073     _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074     _istack = _ustack - 0x1000;
00075
00076     /*
00077     * Exception vector table at fixed hardware address 0xFFFFF80.
00078     * 32 entries -- all unused.
00079     */
00080     .exvectors 0xFFFFF80 : AT(0xFFFFF80)
00081     {
00082         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00089         LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090     } > ROM
00091
00092     /* Fixed reset vector at 0xFFFFF8C */
00093     .fvectors 0xFFFFF8C : AT(0xFFFFF8C)
00094     {
00095         KEEP(*(fvectors))
00096     } > ROM
00097 }

```

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_gptw.h"
```



```
**
**
**
**
**
**
**
```

>

>

>

#define P17PFS 0x0008C14FU /* pin 38 on Tom board */

#define P23PFS 0x0008C153U /* pin 34 on Tom board */

#define PA7_MASK 0x80U

#define PE2PFS 0x0008C172U /* HAL default GTIOC0B -- we disable this */

#define PE5PFS 0x0008C175U /* HAL default GTIOC0A -- we disable this */

```
#define PFS_DISABLED 0x00U
```

```
#define PFS_GPTW_P17_P23 0x1EU
```

```
#define PORT1_PMR 0x0008C061U
```

```
#define PORT2_PMR 0x0008C062U
```

```
#define PORTA_PDR 0x0008C00AU
```

```
#define PORTA_PODR 0x0008C02AU
```

```
#define PORTE_PMR 0x0008C06EU
```

```
#define PWPR 0x0008C11FU
```

```
#define PWPR_LOCK 0x80U /* set B0WI */
```

```
#define PWPR_UNLOCK_STAGE1 0x00U /* clear B0WI */
```

```
#define PWPR_UNLOCK_STAGE2 0x40U /* set PFSWE */
```

```
#define REG8(  
    a)
```

```
(*(volatile uint8_t *) (a))
```

```
enum pwm_const_t : uint32_t
```


```
00070     : uint32_t {  
00071     k_pwm_pcka_hz      = 96000000U,  
00072     k_pwm_freq_hz     = 20000U,  
00073     k_pwm_period_cnt  = 4800U, /* GTPR+1 */  
00074     k_pwm_gtpcr       = 4799U, /* PCKA/freq - 1 */  
00075     k_pwm_duty_pct    = 2400U, /* half of period */  
00076     k_duty_step       = 48U, /* ~1% of period */  
00077     k_duty_hold_loops = 40000U,  
00078 } pwm_const_t;
```

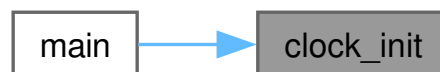
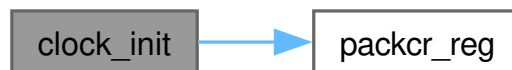
```
void clock_init (  
    void ) [extern]
```

```
00101 {  
00102     volatile rx_system_regs_t *sys = system_regs();  
00103  
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */  
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;  
00106  
00107     /* Step 1: start HOCO if not already running. */  
00108     sys->hococr = k_hococr_running;  
00109  
00110     /* Step 2: wait for HOCO to stabilise. */  
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {  
00112         __asm__ volatile("nop");  
00113     }  
00114  
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */  
00116     sys->pllcr = k_pllcr_hoco_x12;  
00117     sys->pllcr2 = k_pllcr2_enable;  
00118 }
```

```

00119  /* Step 4: wait for PLL to stabilise. */
00120  while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121      __asm__ volatile("nop");
00122  }
00123
00124  /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125  *memwait_reg() = k_memwait_one;
00126
00127  /* Step 6: program all bus dividers. */
00128  sys->sckcr = k_sckcr_value;
00129
00130  /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131  *packcr_reg() = k_packcr_pll_source;
00132
00133  /* Step 8: program the USB clock divider. */
00134  sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136  /* Step 9: switch the system clock source to PLL. */
00137  sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139  /* Step 10: relock PRCR. */
00140  *prcr_reg() = k_rx_prcr_lock;
00141  }

```



```

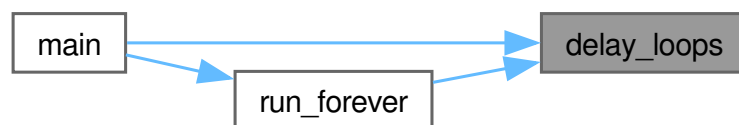
void delay_loops (
    uint32_t n)  [inline], [static]

```

```

00081 {
00082     for (volatile uint32_t i = 0U; i < n; i++) {
00083         __asm__ volatile("nop");
00084     }
00085 }

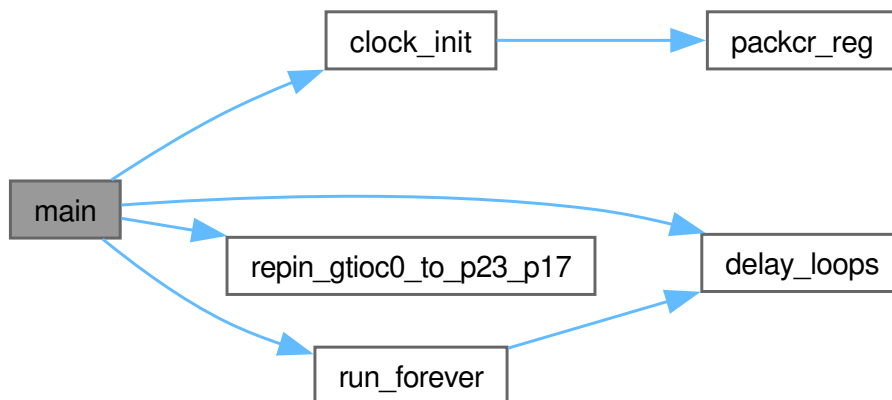
```



```
int main (
```

```
void )
```

```
00155 {
00156     /* Step 1: HOCO x12 -> 192 MHz PLL, PCKA = 96 MHz. */
00157     clock_init();
00158
00159     /* Step 2: Call the HAL to configure GPTW0 as 20 kHz sawtooth PWM with
00160      * 0% initial duty. The HAL:
00161      * - releases the module-stop bit (MSTPCRA.MSTPA7, fixed from MSTPCRC.MSTPC6)
00162      * - computes GTPR = 96M/20k - 1 = 4799 (fixed from 120 MHz assumption)
00163      * - writes GTUDDTYC = UD|UDF for up-count (fixed: was defaulting to 0)
00164      * - writes GTBER = 0 for immediate GTCCRA/B updates (fixed: was single-buf)
00165      * - configures MPC for PE5/PE2 and starts the timer.
00166      * After this call we only need to re-pin outputs to our board (step 3)
00167      * and set the desired duty (step 4). */
00168     const rx_gptw_config_t config = {
00169         .frequency_hz      = k_pwm_freq_hz,
00170         .deadtime_ns       = 0U,
00171         .wave_mode         = k_gptw_wave_saw_pwm,
00172         .enable_complementary = false,
00173         .invert_polarity   = false,
00174     };
00175     rx_err_t err = rx_gptw_init_pwm(k_gptw_channel, &config);
00176     if (err != k_rx_ok) {
00177         /* Fatal: flash LED fast forever so we know something went wrong. */
00178         REG8(PORTA_PDR) |= PA7_MASK;
00179         for (;;) {
00180             REG8(PORTA_PODR) ^= PA7_MASK;
00181             delay_loops(8000U);
00182         }
00183     }
00184
00185     /* Step 3: Re-route the GPTW0 outputs from PE5/PE2 (HAL default) to the
00186      * P23/P17 pins Tom's breakout board exposes. This is legitimate
00187      * board-specific routing, not a bug workaround. */
00188     repin_gtioc0_to_p23_p17();
00189
00190     /* Step 4: Set 50% duty on both outputs. With GTBER=0 these writes take
00191      * effect immediately. */
00192     (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_a, k_pwm_duty_pct);
00193     (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_b, k_pwm_duty_pct);
00194
00195     /* Step 5: Static 50% duty forever (diagnostic). PA7 heartbeat. */
00196     REG8(PORTA_PDR) |= PA7_MASK;
00197     for (;;) {
00198         REG8(PORTA_PODR) ^= PA7_MASK;
00199         delay_loops(200000U);
00200     }
00201     (void)run_forever;
00202
00203     return 0;
00204 }
```



```

void repin_gtioc0_to_p23_p17 (
    void ) [static]

00093 {
00094     /* Undo HAL's peripheral mode on PE5/PE2 first, before changing PFS. */
00095     REG8(PORTE_PMR) &= (uint8_t)~((1U « 5) | (1U « 2));
00096
00097     /* Enable PFS writes. */
00098     REG8(PWPR) = PWPR_UNLOCK_STAGE1;
00099     REG8(PWPR) = PWPR_UNLOCK_STAGE2;
00100
00101     /* Disable PE5/PE2 routing and enable P23/P17 routing. */
00102     REG8(PE5PFS) = PFS_DISABLED;
00103     REG8(PE2PFS) = PFS_DISABLED;
00104     REG8(P23PFS) = PFS_GPTW_P17_P23;
00105     REG8(P17PFS) = PFS_GPTW_P17_P23;
00106
00107     /* Re-protect PFS. */
00108     REG8(PWPR) = PWPR_UNLOCK_STAGE1;
00109     REG8(PWPR) = PWPR_LOCK;
00110
00111     /* Switch P23/P17 to peripheral mode. */
00112     REG8(PORT2_PMR) |= (uint8_t)(1U « 3); /* P23 */
00113     REG8(PORT1_PMR) |= (uint8_t)(1U « 7); /* P17 */
00114 }

```

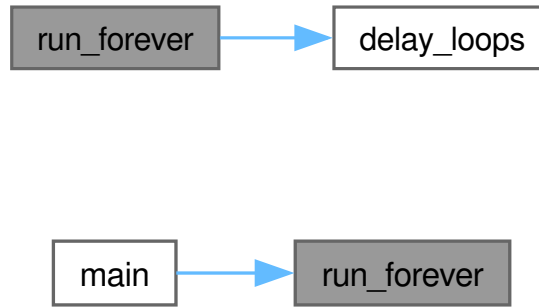


```

void run_forever (
    void ) [static]

00119 {
00120     REG8(PORTA_PDR) |= PA7_MASK;
00121     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00122
00123     uint32_t duty = 0U;
00124     uint32_t direction = 1U;
00125
00126     for (;;) {
00127         /* Same duty on both A and B (both outputs mirror each other for scope
00128          * visibility). For a real H-bridge one output would be fixed direction
00129          * and the other the variable-duty PWM. */
00130         (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_a, duty);
00131         (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_b, duty);
00132
00133         REG8(PORTA_PODR) ^= PA7_MASK;
00134
00135         delay_loops(k_duty_hold_loops);
00136
00137         if (direction != 0U) {
00138             duty += (uint32_t)k_duty_step;
00139             if (duty >= (uint32_t)k_pwm_period_cnt) {
00140                 duty = (uint32_t)k_pwm_period_cnt;
00141                 direction = 0U;
00142             }
00143         } else {
00144             if (duty <= (uint32_t)k_duty_step) {
00145                 duty = 0U;
00146                 direction = 1U;
00147             } else {
00148                 duty -= (uint32_t)k_duty_step;
00149             }
00150         }
00151     }
00152 }

```



```
00001 /**
00002  * @file main.c
00003  * @brief HAL-driven 20 kHz PWM on GPTW0 channel 0 with outputs on P23/P17.
00004  *
00005  * @details
00006  * Standalone firmware that exercises the STAR libs/rx_hal/src/rx_gptw.c HAL
00007  * directly to drive Tom's breakout-board PWM pins:
00008  *
00009  * - P23 / GTIOC0A -> Tom header pin 34 ("IN2", scope Ch2 blue)
00010  * - P17 / GTIOC0B -> Tom header pin 38 ("IN1", scope Ch1 orange)
00011  *
00012  * These are NOT the HAL's default PE5/PE2 routing. The HAL configures MPC
00013  * internally for PE5/PE2, so after rx_gptw_init_pwm() we undo that routing
00014  * and re-pin GTIOC0A/B on P23/P17.
00015  *
00016  * Clock path (from pwm_test's clock.c):
00017  * - HOCO 16 MHz x 12 = 192 MHz PLL
00018  * - ICLK 96 MHz, PCKA 96 MHz
00019  *
00020  * The STAR HAL previously contained four bugs that this test first
00021  * exposed: (1) wrong module-stop bit (MSTPCRC.MSTPC6 instead of
00022  * MSTPCRA.MSTPA7), (2) missing GTUDDTYC write (counter defaulted to
00023  * down-count), (3) default single-buffer GTBER (duty writes swallowed),
00024  * (4) hard-coded 120 MHz PCKA (real PCKA = 96 MHz). Those bugs are now
00025  * fixed in rx_gptw.c / rx72n_gptw_regs.h / rx72n_clock.h, so the only
00026  * board-specific adjustment left here is the PE5/PE2 -> P23/P17 re-pin.
00027  *
00028  * SPDX-License-Identifier: MIT
00029  * @copyright Copyright (c) 2026 Locked Inc.
00030  */
00031
00032 #include <stdbool.h>
00033 #include <stdint.h>
00034
00035 #include "rx_gptw.h"
00036
00037 extern void clock_init(void);
00038
00039 #define REG8(a) (*(volatile uint8_t *) (a))
00040
00041 /* MPC pin function select */
00042 #define PWPR 0x0008C11FU
00043 #define P17PFS 0x0008C14FU /* pin 38 on Tom board */
00044 #define P23PFS 0x0008C153U /* pin 34 on Tom board */
00045 #define PE2PFS 0x0008C172U /* HAL default GTIOC0B -- we disable this */
00046 #define PE5PFS 0x0008C175U /* HAL default GTIOC0A -- we disable this */
00047
00048 #define PWPR_UNLOCK_STAGE1 0x00U /* clear B0WI */
00049 #define PWPR_UNLOCK_STAGE2 0x40U /* set PFSWE */
00050 #define PWPR_LOCK 0x80U /* set B0WI */
00051
00052 /* P17/P23 use PSEL = 0b011110 = 0x1E for GTIOC0A/B per RX72N HW manual
00053  * Ch.23 Table 23.4 (P17) and Table 23.6 (P23). 0x14 is the correct PSEL
00054  * for GTIOC on the Port E family (PE5/PE2), but Ports 1/2 use a
00055  * different encoding. */
00056 #define PFS_GPTW_P17_P23 0x1EU
00057 #define PFS_DISABLED 0x00U
00058
00059 /* PORT peripheral mode / direction */
00060 #define PORT1_PMR 0x0008C061U
00061 #define PORT2_PMR 0x0008C062U
00062 #define PORTE_PMR 0x0008C06EU
00063
00064 /* Heartbeat LED on PA7 */
00065 #define PORTA_PDR 0x0008C00AU
00066 #define PORTA_PODR 0x0008C02AU
00067 #define PA7_MASK 0x80U
00068
```

```

00069 /* PWM constants for 20 kHz at PCKA = 96 MHz */
00070 typedef enum : uint32_t {
00071     k_pwm_pcka_hz      = 96000000U,
00072     k_pwm_freq_hz     = 20000U,
00073     k_pwm_period_cnt  = 4800U, /* GTPR+1 */
00074     k_pwm_gtp         = 4799U, /* PCKA/freq - 1 */
00075     k_pwm_duty_pct    = 2400U, /* half of period */
00076     k_duty_step       = 48U, /* ~1% of period */
00077     k_duty_hold_loops = 40000U,
00078 } pwm_const_t;
00079
00080 static inline void delay_loops(uint32_t n)
00081 {
00082     for (volatile uint32_t i = 0U; i < n; i++) {
00083         __asm__ volatile("nop");
00084     }
00085 }
00086
00087 /* The HAL's rx_gptw_init_pwm configures MPC for PE5/PE2 (HAL's default
00088 * Motor 0 pins). We need GTIOC0A/B on P23/P17 instead. This function:
00089 * 1. Disables PE5/PE2 peripheral output (clear PMR + clear PFS).
00090 * 2. Enables P23/P17 as GTIOC0A/B (PFS = 0x14, PMR = 1).
00091 * Must be called AFTER rx_gptw_init_pwm() returns. */
00092 static void repin_gtioc0_to_p23_p17(void)
00093 {
00094     /* Undo HAL's peripheral mode on PE5/PE2 first, before changing PFS. */
00095     REG8(PORTE_PMR) &= (uint8_t)~((1U << 5) | (1U << 2));
00096
00097     /* Enable PFS writes. */
00098     REG8(PWPR) = PWPR_UNLOCK_STAGE1;
00099     REG8(PWPR) = PWPR_UNLOCK_STAGE2;
00100
00101     /* Disable PE5/PE2 routing and enable P23/P17 routing. */
00102     REG8(PE5PFS) = PFS_DISABLED;
00103     REG8(PE2PFS) = PFS_DISABLED;
00104     REG8(P23PFS) = PFS_GPTW_P17_P23;
00105     REG8(P17PFS) = PFS_GPTW_P17_P23;
00106
00107     /* Re-protect PFS. */
00108     REG8(PWPR) = PWPR_UNLOCK_STAGE1;
00109     REG8(PWPR) = PWPR_LOCK;
00110
00111     /* Switch P23/P17 to peripheral mode. */
00112     REG8(PORT2_PMR) |= (uint8_t)(1U << 3); /* P23 */
00113     REG8(PORT1_PMR) |= (uint8_t)(1U << 7); /* P17 */
00114 }
00115
00116 /* Drive the PA7 heartbeat LED and sweep the PWM duty cycle 0..100..0 forever.
00117 * Uses rx_gptw_set_duty_raw for speed (avoids float dispatch). */
00118 static void run_forever(void)
00119 {
00120     REG8(PORTA_PDR) |= PA7_MASK;
00121     REG8(PORTA_PODR) &= (uint8_t)~PA7_MASK;
00122
00123     uint32_t duty = 0U;
00124     uint32_t direction = 1U;
00125
00126     for (;;) {
00127         /* Same duty on both A and B (both outputs mirror each other for scope
00128          * visibility). For a real H-bridge one output would be fixed direction
00129          * and the other the variable-duty PWM. */
00130         (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_a, duty);
00131         (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_b, duty);
00132
00133         REG8(PORTA_PODR) ^= PA7_MASK;
00134
00135         delay_loops(k_duty_hold_loops);
00136
00137         if (direction != 0U) {
00138             duty += (uint32_t)k_duty_step;
00139             if (duty >= (uint32_t)k_pwm_period_cnt) {
00140                 duty = (uint32_t)k_pwm_period_cnt;
00141                 direction = 0U;
00142             }
00143         } else {
00144             if (duty <= (uint32_t)k_duty_step) {
00145                 duty = 0U;
00146                 direction = 1U;
00147             } else {
00148                 duty -= (uint32_t)k_duty_step;
00149             }
00150         }
00151     }
00152 }
00153
00154 int main(void)
00155 {

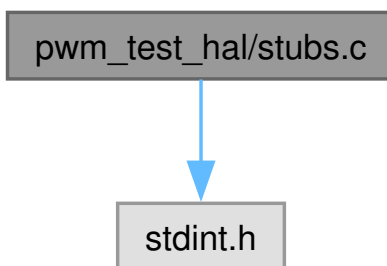
```

```

00156  /* Step 1: HOCO x12 -> 192 MHz PLL, PCKA = 96 MHz. */
00157  clock_init();
00158
00159  /* Step 2: Call the HAL to configure GPTW0 as 20 kHz sawtooth PWM with
00160  * 0% initial duty. The HAL:
00161  * - releases the module-stop bit (MSTPCRA.MSTPA7, fixed from MSTPCRC.MSTPC6)
00162  * - computes GTPR = 96M/20k - 1 = 4799 (fixed from 120 MHz assumption)
00163  * - writes GTUDDTYC = UD|UDF for up-count (fixed: was defaulting to 0)
00164  * - writes GTBER = 0 for immediate GTCCRA/B updates (fixed: was single-buf)
00165  * - configures MPC for PE5/PE2 and starts the timer.
00166  * After this call we only need to re-pin outputs to our board (step 3)
00167  * and set the desired duty (step 4). */
00168  const rx_gptw_config_t config = {
00169      .frequency_hz      = k_pwm_freq_hz,
00170      .deadtime_ns       = 0U,
00171      .wave_mode         = k_gptw_wave_saw_pwm,
00172      .enable_complementary = false,
00173      .invert_polarity    = false,
00174  };
00175  rx_err_t err = rx_gptw_init_pwm(k_gptw_channel, &config);
00176  if (err != k_rx_ok) {
00177      /* Fatal: flash LED fast forever so we know something went wrong. */
00178      REG8(PORTA_PDR) |= PA7_MASK;
00179      for (;;) {
00180          REG8(PORTA_PODR) ^= PA7_MASK;
00181          delay_loops(8000U);
00182      }
00183  }
00184
00185  /* Step 3: Re-route the GPTW0 outputs from PE5/PE2 (HAL default) to the
00186  * P23/P17 pins Tom's breakout board exposes. This is legitimate
00187  * board-specific routing, not a bug workaround. */
00188  repin_gtioc0_to_p23_p17();
00189
00190  /* Step 4: Set 50% duty on both outputs. With GTBER=0 these writes take
00191  * effect immediately. */
00192  (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_a, k_pwm_duty_pct);
00193  (void)rx_gptw_set_duty_raw(k_gptw_channel, k_gptw_output_b, k_pwm_duty_pct);
00194
00195  /* Step 5: Static 50% duty forever (diagnostic). PA7 heartbeat. */
00196  REG8(PORTA_PDR) |= PA7_MASK;
00197  for (;;) {
00198      REG8(PORTA_PODR) ^= PA7_MASK;
00199      delay_loops(200000U);
00200  }
00201  (void)run_forever;
00202
00203  return 0;
00204 }

```

```
#include <stdint.h>
```



*

*

*

```
void __tx_thread_context_restore (
    void )
```

```
00045 {}
```

```
void __tx_thread_context_save (
    void )
```

```
00044 {}
```

```
void cmt0_isr (
    void )
```

```
00046 {}
```

```
void rx_log_uart_putc (
    char c)
```

```
00028 { (void)c; }
```

```
void rx_log_uart_puthex (  
    uint32_t v,  
    uint8_t d)
```

```
00032 { (void)v; (void)d; }
```

```
void rx_log_uart_putint (  
    int32_t v)
```

```
00030 { (void)v; }
```

```
void rx_log_uart_puts (  
    const char * s)
```

```
00029 { (void)s; }
```

```
void rx_log_uart_putuint (  
    uint32_t v)
```

```
00031 { (void)v; }
```

```
void uart_debug_putc (  
    char c)
```

```
00036 { (void)c; }
```

```
void uart_debug_puthex (  
    uint32_t v,  
    uint8_t d)
```

```
00040 { (void)v; (void)d; }
```

```
void uart_debug_putint (  
    int32_t v)
```

```
00038 { (void)v; }
```

```
void uart_debug_puts (  
    const char * s)
```

```
00037 { (void)s; }
```

```
void uart_debug_putuint (  
    uint32_t v)
```

```
00039 { (void)v; }
```

```
00001 /**
00002  * @file stubs.c
00003  * @brief Stubs for ThreadX, log, and ISR symbols pulled in by the STAR HAL.
00004  *
00005  * @details
00006  * The HAL pulls in a set of symbols we don't need for a minimal build:
00007  *
00008  * - rx_log_uart_puts/putc/putint/putuint/puthex: hardware log-ring API used
00009  *   by rx_log.h in non-simulator builds. rx_log_uart.c depends on ThreadX
00010  *   (tx_api.h), so we stub them out to void-return.
00011  *
00012  * - uart_debug_*: low-level raw-UART debug writers referenced by
00013  *   rx_check.h's internal_rx_fatal_error. Stub to void-return.
00014  *
00015  * - _tx_thread_context_save/restore: referenced by vectors.S for the
00016  *   SWINT handler. Not exercised in this bare-metal test.
00017  *
00018  * - cmt0_isr: referenced by vectors.S for the 100 Hz ThreadX tick vector.
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 /* ===== rx_log_uart_* stubs ===== */
00027
00028 void rx_log_uart_putc(char c)          { (void)c; }
00029 void rx_log_uart_puts(const char* s)   { (void)s; }
00030 void rx_log_uart_putint(int32_t v)     { (void)v; }
00031 void rx_log_uart_putuint(uint32_t v)   { (void)v; }
00032 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00033
00034 /* ===== uart_debug_* stubs (used by rx_check.h fatal-error helper) ===== */
00035
00036 void uart_debug_putc(char c)          { (void)c; }
00037 void uart_debug_puts(const char* s)   { (void)s; }
00038 void uart_debug_putint(int32_t v)     { (void)v; }
00039 void uart_debug_putuint(uint32_t v)   { (void)v; }
00040 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00041
00042 /* ===== ThreadX / ISR stubs referenced by vectors.S ===== */
00043
00044 void _tx_thread_context_save(void) {}
00045 void _tx_thread_context_restore(void) {}
00046 void cmt0_isr(void) {}
```
